WEEK 9: SALES AND CUSTOMER ANALYTICS SYSTEM

Scenario:

The Art Workshop wants to analyze its sales data to understand customer purchasing behavior, product performance,and revenue trends. The management needs reports to identify top customers,best-selling products, and profitable categories.

1.Apply SQL Aggregate Functions:

SQL Aggregate functions perform calculations on a set of multiple records and return a single summarized value or a result for each group.

COUNT() – The COUNT() function counts the number of records in a table.
SUM() – The SUM() function calculates the total of a numeric column.
AVG() – The AVG() function calculates the average values of a numeric column.
MAX() – The MAX() function returns the highest value in a column.
MIN() – The MIN() function returns the lowest value in a column.

2.Generate Total Sales Reports:

Tasks: Write SQL queries to calculate:

1.TOTAL NUMBER OF ORDERS PLACED

```
SQL> SELECT COUNT(*)AS ORDERS_ID FROM ORDERS;

 ORDERS_ID
----------
        10
```

2.TOTAL REVENUE GENERATED FROM SALES

```
SQL> SELECT SUM(TOTAL_AMOUNT)FROM ORDERS;

SUM(TOTAL_AMOUNT)
-----------------
            20100
```

```
SQL> SELECT SUM(TOTAL_AMOUNT)AS REVENUE FROM ORDERS;

   REVENUE
----------
     20100
```

3.AVERAGE ORDER VALUE

```
SQL> SELECT AVG(TOTAL_AMOUNT)FROM ORDERS;

AVG(TOTAL_AMOUNT)
-----------------
             2010

SQL> SELECT AVG(TOTAL_AMOUNT)AS ORDER_VALUE FROM ORDERS;

ORDER_VALUE
-----------
       2010
```

❖ AVERAGE ORDER VALUE OF EACH ORDER AT A TIME

```
SQL> SELECT ORDER_ID,AVG(TOTAL_AMOUNT)FROM ORDERS GROUP BY ORDER_ID;

ORDER_I AVG(TOTAL_AMOUNT)
------- -----------------
0001                2800
0002                1500
0003                3000
0004                4000
0005                1000
0006                1200
0007                5000
0008                 800
0009                 300
0010                 500

10 rows selected.
```

4.HIGHEST AND LOWEST ORDER AMOUNT

```
SQL> SELECT MIN(TOTAL_AMOUNT)FROM ORDERS;

MIN(TOTAL_AMOUNT)
-----------------
              300

SQL> SELECT MAX(TOTAL_AMOUNT)FROM ORDERS;

MAX(TOTAL_AMOUNT)
-----------------
             5000
```

```
SQL> SELECT MIN(TOTAL_AMOUNT)AS LOWEST_ORDERS FROM ORDERS;

LOWEST_ORDERS
-------------
          300

SQL> SELECT MAX(TOTAL_AMOUNT)AS HIGHEST_ORDERS FROM ORDERS;

HIGHEST_ORDERS
--------------
          5000
```

5.TOTAL SALES FOR A SPECIFIC PERIOD

```
SQL> SELECT COUNT(ORDER_ID) AS TOTAL_ORDERS,SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE,AVG(TOTAL_AMOUNT) AS AVG_ORDER_VALUE FROM ORDERS;

TOTAL_ORDERS TOTAL_REVENUE AVG_ORDER_VALUE
------------ ------------- ---------------
          10         20100            2010
```

3.CUSTOMER PURCHASE ANALYSIS

1.TOTAL NUMBER OF ORDERS PLACED BY EACH CUSTOMER

```
SQL> SELECT COUNT(*),CUSTOMER_ID FROM ORDERS GROUP BY CUSTOMER_ID;

  COUNT(*) CUSTOMER_ID
---------- -----------
         1           1
         1           6
         1           2
         1           4
         1           5
         1           8
         1           3
         1           7
         1           9
         1          10

10 rows selected.
```

2.TOTAL AMOUNT SPENT BY EACH CUSTOMER

```
SQL> SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) FROM ORDERS GROUP BY CUSTOMER_ID;

CUSTOMER_ID SUM(TOTAL_AMOUNT)
----------- -----------------
          1              2800
          6              1200
          2              1500
          4              4000
          5              1000
          8               800
          3              3000
          7              5000
          9               300
         10               500

10 rows selected.
```

3.AVERAGE SPENDING PER CUSTOMER

```
SQL> SELECT CUSTOMER_ID,AVG(TOTAL_AMOUNT) FROM ORDERS GROUP BY CUSTOMER_ID;

CUSTOMER_ID AVG(TOTAL_AMOUNT)
----------- -----------------
          1              2800
          6              1200
          2              1500
          4              4000
          5              1000
          8               800
          3              3000
          7              5000
          9               300
         10               500

10 rows selected.
```

4.CUSTOMER WITH MAXIMUM PURCHASE AMOUNT

```
SQL> SELECT CUSTOMER_ID,TOTAL_AMOUNT AS MAX_PURCHASE_AMOUNT FROM ORDERS WHERE TOTAL_AMOUNT=(SELECT MAX(TOTAL_AMOUNT) FROM ORDERS);

CUSTOMER_ID MAX_PURCHASE_AMOUNT
----------- -------------------
          7                5000
```

5.CUSTOMER WITH FEWER PURCHASES

```
SQL> SELECT CUSTOMER_ID,TOTAL_AMOUNT AS MIN_PURCHASE_AMOUNT FROM ORDERS WHERE TOTAL_AMOUNT=(SELECT MIN(TOTAL_AMOUNT) FROM ORDERS);

CUSTOMER_ID MIN_PURCHASE_AMOUNT
----------- -------------------
          9                 300
```

4.FIND TOP CUSTOMERS BASED ON PURCHASE AMOUNT

1.TOP 5 CUSTOMERS BASED ON TOTAL SPENDING

```
SQL> SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) FROM ORDERS GROUP BY CUSTOMER_ID ORDER BY SUM(TOTAL_AMOUNT) DESC;

CUSTOMER_ID SUM(TOTAL_AMOUNT)
----------- -----------------
          7              5000
          4              4000
          3              3000
          1              2800
          2              1500
          6              1200
          5              1000
          8               800
         10               500
          9               300

10 rows selected.
```

2.CUSTOMER WITH MAXIMUM NUMBER OF ORDERS

```
SQL> SELECT COUNT(*) AS NO_OF_ORDERS,CUSTOMER_ID FROM ORDERS GROUP BY CUSTOMER_ID ORDER BY NO_OF_ORDERS DESC;

NO_OF_ORDERS CUSTOMER_ID
------------ -----------
           1           1
           1           6
           1           2
           1           4
           1          10
           1           8
           1           3
           1           7
           1           9
           1           5

10 rows selected.
```

3.FREQUENT CUSTOMERS

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.FIRST_NAME,
  4      C.LAST_NAME,
  5      COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  6  FROM CUSTOMER C
  7  JOIN ORDERS O
  8      ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9  GROUP BY
 10      C.CUSTOMER_ID,
 11      C.FIRST_NAME,
 12      C.LAST_NAME
 13  ORDER BY TOTAL_ORDERS DESC;

CUSTOMER_ID FIRST_NAME                                         LAST_NAME                                          TOTAL_ORDERS
----------- -------------------------------------------------- -------------------------------------------------- ------------
          1 Arun                                               Kumar                                                         1
          3 Karthik                                            Raj                                                           1
          8 Pooja                                              Nair                                                          1
         10 Harini                                             Krishnan                                                      1
          2 Meena                                              Ravi                                                          1
          9 Sanjay                                             Das                                                           1
          4 Swetha                                             Mohan                                                         1
          5 Imran                                              Ali                                                           1
          7 Rohan                                              Sharma                                                        1
          6 Keerthi                                            Suresh                                                        1

10 rows selected.
```

4.HIGH VALUE CUSTOMERS

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.FIRST_NAME,
  4      C.LAST_NAME,
  5      SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENT
  6  FROM CUSTOMER C
  7  JOIN ORDERS O
  8      ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9  GROUP BY
 10      C.CUSTOMER_ID,
 11      C.FIRST_NAME,
 12      C.LAST_NAME
 13  ORDER BY TOTAL_SPENT DESC;

CUSTOMER_ID FIRST_NAME                                         LAST_NAME                                          TOTAL_SPENT
----------- -------------------------------------------------- -------------------------------------------------- -----------
          7 Rohan                                              Sharma                                                    5000
          4 Swetha                                             Mohan                                                     4000
          3 Karthik                                            Raj                                                       3000
          1 Arun                                               Kumar                                                     2800
          2 Meena                                              Ravi                                                      1500
          6 Keerthi                                            Suresh                                                    1200
          5 Imran                                              Ali                                                       1000
          8 Pooja                                              Nair                                                       800
         10 Harini                                             Krishnan                                                   500
          9 Sanjay                                             Das                                                        300

10 rows selected.
```

5.IDENTIFY BEST-SELLING PRODUCTS:

1.PRODUCTS WITH MAXIMUM SALES QUANTITY

```
SQL> SELECT PRODUCT_NAME,STOCK_QUANTITY FROM PRODUCT ORDER BY STOCK_QUANTITY DESC;

PRODUCT_NAME         STOCK_QUANTITY
-------------------- --------------
Face Cream                       50
Face Wash                        40
Lipstick                         30
Kids Shoes                        2
Laptop                            1
```

2.PRODUCTS GENERATING HIGHEST REVENUE

```
SQL> SELECT PRODUCT_NAME,PRODUCT_ID,SUM(STOCK_QUANTITY * PRICE) AS REVENUE FROM PRODUCT GROUP BY PRODUCT_NAME,PRODUCT_ID ORDER BY REVENUE DESC;

PRODUCT_NAME         PRODUCT    REVENUE
-------------------- ------- ----------
Face Cream           P000003      12500
Lipstick             P000004      10500
Face Wash            P000005       7200
Kids Shoes           P200002         30
Laptop               P100001         20
```

3.LEAST SELLING PRODUCT

```
SQL> SELECT PRODUCT_NAME,PRODUCT_ID,SUM(PRICE * STOCK_QUANTITY) AS SELLING_PRICE FROM PRODUCT GROUP BY PRODUCT_NAME,PRODUCT_ID ORDER BY SELLING_PRICE ASC;

PRODUCT_NAME         PRODUCT SELLING_PRICE
-------------------- ------- -------------
Laptop               P100001            20
Kids Shoes           P200002            30
Face Wash            P000005          7200
Lipstick             P000004         10500
Face Cream           P000003         12500
```

4.PRODUCTS REQUIRING PROMOTION

```
SQL> SELECT PRODUCT_ID,PRODUCT_NAME,CATEGORY_ID,PRICE,STOCK_QUANTITY FROM PRODUCT WHERE STOCK_QUANTITY < 10 ORDER BY STOCK_QUANTITY ASC;

PRODUCT PRODUCT_NAME         CATEGOR      PRICE STOCK_QUANTITY
------- -------------------- ------- ---------- --------------
P100001 Laptop               55000           20              1
P200002 Kids Shoes           1500            15              2
```

6.PERFORM CATEGORY-WISE SALES ANALYSIS

1.TOTAL SALES GENERATED BY EACH CATEGORY

```
SQL> SELECT C.CATEGORY_ID,C.CATEGORY_NAME,SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_SALES FROM CATEGORY C JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP
BY C.CATEGORY_ID,C.CATEGORY_NAME ORDER BY TOTAL_SALES DESC;

no rows selected
```

2.NUMBER OF PRODUCTS SOLD IN EACH CATEGORY

```
SQL> SELECT C.CATEGORY_NAME,COUNT(OD.PRODUCT_ID) AS TOTAL_PRODUCTS_SOLD FROM CATEGORY C JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY CATEGORY_N
AME;

no rows selected
```

3.HIGHEST REVENUE-GENERATING CATEGORY

```
SQL> SELECT SUM(O.TOTAL_AMOUNT) AS TOTAL_SALES,P.PRODUCT_NAME FROM ORDERS O JOIN ORDER_DETAILS OD ON O.ORDER_ID = OD.ORDER_ID JOIN PRODUCT P ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_NAME ORDER BY TOT
AL_SALES DESC;

TOTAL_SALES PRODUCT_NAME
----------- --------------------
       7500 Kids Shoes
       6000 Laptop
```

4.AVERAGE SALES PER CATEGORY

```
SQL> SELECT AVG(O.TOTAL_AMOUNT) AS AVG_SALES,P.PRODUCT_NAME FROM ORDERS O JOIN ORDER_DETAILS OD ON O.ORDER_ID = OD.ORDER_ID JOIN PRODUCT P ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_NAME;

 AVG_SALES PRODUCT_NAME
---------- --------------------
      1500 Kids Shoes
      1500 Laptop
```

7.GENERATE BUSINESS ANALYTICS REPORT

REPORT 1: SALES PERFORMANCE REPORT

- **Total sales**
- **Number of orders**
- **Average order value**
- **Highest order value**

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES,SUM(ORDER_ID) AS NUMBER_OF_ORDERS,AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE,MAX(T
OTAL_AMOUNT) AS HIGHEST_ORDER_VALUE FROM ORDERS ORDER BY TOTAL_AMOUNT DESC;

TOTAL_SALES NUMBER_OF_ORDERS AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
----------- ---------------- ------------------- -------------------
      20100               55                2010                5000
```

REPORT 2:CUSTOMER ANALYTICS REPORT

- **Customer name**
- **Number of orders**
- **Total spending**
- **Purchase frequency**

```
SQL> SELECT
  2      C.FIRST_NAME AS CUSTOMER_NAME,
  3      COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS,
  4      SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING,
  5      COUNT(O.ORDER_ID) AS PURCHASE_FREQUENCY
  6  FROM CUSTOMER C
  7  JOIN ORDERS O
  8  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9  GROUP BY C.CUSTOMER_ID, C.FIRST_NAME;

CUSTOMER_NAME                                      NUMBER_OF_ORDERS TOTAL_SPENDING PURCHASE_FREQUENCY
-------------------------------------------------- ---------------- -------------- ------------------
Imran                                                             1           1000                  1
Pooja                                                             1            800                  1
Harini                                                            1            500                  1
Arun                                                              1           2800                  1
Karthik                                                           1           3000                  1
Rohan                                                             1           5000                  1
Swetha                                                            1           4000                  1
Sanjay                                                            1            300                  1
Meena                                                             1           1500                  1
Keerthi                                                           1           1200                  1

10 rows selected.
```

REPORT 3:PRODUCT PERFORMANCE REPORT

- **Product name**
- **Quantity sold**
- **Revenue generated**

```
SQL> SELECT P.PRODUCT_NAME,SUM(OD.QUANTITY) AS QUANTITY_SOLD,SUM(OD.QUANTITY * P.PRICE) AS REVENUE_GENERATED FROM PROD
UCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_NAME = OD.PRODUCT_ID GROUP BY P.PRODUCT_NAME;

no rows selected
```

REPORT 4 : CATEGORY ANALYSIS REPORT

- **Category name**
- **Total products sold**
- **Total revenue**

```
SQL> SELECT C.CATEGORY_NAME,SUM(OD.QUANTITY) AS TOTAL_PRODUCTS_SOLD,SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE FROM CATEGORY C INNER JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID INNER JOIN ORDER_DETAILS O
D ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY C.CATEGORY_NAME;

no rows selected
```